Chapter 9

Iterative Methods of Solution

Iterative or indirect method are based on reformulating $A\mathbf{x} = \mathbf{b}$ as

$$\mathbf{x}^{(n+1)} = M\mathbf{x}^{(n)} + \mathbf{c}$$

where n refers to the iteration number, M is known as the iteration matrix and $\mathbf{c}$ is a new vector formed by the reformulation process. This approach requires that $\mathbf{x}^{(n)}$ converges to $\mathbf{x}$, the exact solution, i.e.

$$\mathbf{x} = \lim_{n \to \infty} \mathbf{x}^{(n)}$$

Convergence depends on the choice of M and should not depend critically upon the choice of $\mathbf{x}^{(0)}$. Ideally, the choice of $\mathbf{x}^{(0)}$ should be based on *a priori* information on $\mathbf{x}$ but, in practice, we can set $\mathbf{x}^{(0)} = 0$.

Advantages of Iterative Methods

(i) The iteration matrix is virtually unaltered by most methods.

(ii) The methods are generally self correcting.

(iii) The methods are generally simple to program and require little additional storage.

(iv) The methods lend themselves to the application of parallel processing.

Disadvantages of Iterative Methods

(i) In general, there is no guaranty of convergence.

(ii) Even when the method converges, the number of iterations can be large.

(iii) The computing time can be large if many iteration are required and the system of equations is large.

As a general rule of thumb, iterative methods are recommended only for large linear systems of equations $A\mathbf{x} = \mathbf{b}$ when A is a large sparse matrix. Systems of this type, frequently occur in numerical solutions to partial differential equations using finite difference analysis and (to a lesser extent) finite element analysis. They also occur in cases when the impulse response function of a system is characterized by a matrix which is banded and sparse.

The types of iterative techniques that are discussed in this chapter include:

Jacobi's method,
the Gauss-Seidel method,
the relaxation method,
The conjugate gradient method which is a semi-iterative technique.

The conditions for convergence (which are discussed later on in this chapter) are as follows:

(i) A sufficient condition for convergence is that A is strictly diagonally dominant, i.e.

$$|a_{ii}| > \sum_{\substack{j=1\\ j\neq i}}^{n} |a_{ij}|.$$

(ii) A necessary and sufficient condition for convergence is that

$$\rho(M) < 1$$

where $\rho(M)$ is the 'spectral radius', given by

$$\rho(M) = \max_i |\lambda_i|.$$

Here, λ_i are the eigenvalues of M. The largest eigenvalue of the iteration matrix must therefore be less than unity to ensure convergence.

9.1 Basic Ideas

Consider the following system of equations:

$$\begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}.$$

We can rewrite this set of equations as

$$x_1 = \frac{1}{a_{11}}(b_1 - a_{12}x_2 - \dots - a_{1n}x_n),$$

$$x_2 = \frac{1}{a_{22}}(b_2 - a_{21}x_1 - \dots - a_{2n}x_n),$$

$$\vdots$$

$$x_n = \frac{1}{a_{nn}}(b_n - a_{n1}x_1 - \dots - a_{n,n-1}x_{n-1}).$$

Clearly this requires that $a_{ii} \neq 0$, i.e. the diagonal elements are non zero.

9.1.1 Jacobi's Method

Jacobi's method involves the most basic iteration process of the type:

$$x_1^{(k+1)} = \frac{1}{a_{11}}(b_1 - a_{12}x_2^{(k)} - ... - a_{1n}x_n^{(k)}),$$

$$x_2^{(k+1)} = \frac{1}{a_{22}}(b_2 - a_{21}x_1^{(k)} - ... - a_{2n}x_n^{(k)}),$$

$$\vdots$$

$$x_n^{(k+1)} = \frac{1}{a_{nn}}(b_n - a_{n1}x_1^{(k)} - ... - a_{n(n-1)}x_{n-1}^{(k)}).$$

9.1.2 The Gauss-Seidel Method

The Gauss-Seidel method involves updating the sub-diagonal elements as the computation proceeds. The iteration process is

$$x_1^{(k+1)} = \frac{1}{a_{11}}(b_1 - a_{12}x_2^{(k)} - ... - a_{1n}x_n^{(k)}),$$

$$x_2^{(k+1)} = \frac{1}{a_{22}}(b_2 - a_{21}x_1^{(k+1)} - ... - a_{2n}x_n^{(k)}),$$

$$\vdots$$

$$x_n^{(k+1)} = \frac{1}{a_{nn}}(b_n - a_{n1}x_1^{(k+1)} - ... - a_{n(n-1)}x_{n-1}^{(k+1)}).$$

9.1.3 The Relaxation or Chebyshev Method

The relaxation method involves using a 'relaxation parameter' to 'tune' the system in order to reduce the number of iterations required. The iteration process is

$$x_1^{(k+1)} = x_1^{(k)} + \frac{\omega}{a_{11}}(b_1 - a_{12}x_2^{(k)} - ... - a_{1n}x_n^{(k)} - a_{11}x_1^{(k)}),$$

$$x_2^{(k+1)} = x_2^{(k)} + \frac{\omega}{a_{22}}(b_2 - a_{21}x_1^{(k+1)} - ... - a_{2n}x_n^{(k)} - a_{22}x_2^{(k)}),$$

$$\vdots$$

$$x_n^{(k+1)} = x_n^{(k)} + \frac{\omega}{a_{nn}}(b_n - a_{n1}x_1^{(k+1)} - ... - a_{n(n-1)}x_{n-1}^{(k+1)} - a_{nn}x_n^{(k)}).$$

where ω is called the 'relaxation parameter'. The value of this parameter affects (increases) the rate of convergence. There are two cases in which $1 < \omega < 2$ giving the so called 'Successive-Over-Relaxation' or SOR method and the case when $0 < \omega < 1$ known as the 'Successive-Under-Relaxation' or SUR method.

9.2 Iterative Methods

In the previous section the basic ideas were introduced. In this section, we develop the approach further. We start with $\mathbf{b} = A\mathbf{x}$ where A is a $n \times n$ matrix and write this equation in the form

$$b_i = \sum_{j=1}^{n} a_{ij}x_j = \sum_{\substack{j=1 \\ j\neq i}}^{n} a_{ij}x_j + a_{ii}x_i.$$

Rearranging,

$$x_i = \frac{1}{a_{ii}}\left(b_i - \sum_{\substack{j=1 \\ j\neq i}}^{n} a_{ij}x_j\right)$$

and iterating,

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{\substack{j=1 \\ j\neq i}}^{n} a_{ij}x_j^{(k)}\right), \quad k = 1, 2, ...$$

where

$$x_i^{(1)} = \frac{b_i}{a_{ii}}$$

which is equivalent to setting $x_i^{(0)} = 0 \forall i$. The above iterative scheme is Jacobi's method. Gauss-Seidel iteration is given by

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k)}\right)$$

and the relaxation method is compounded in the result

$$x_i^{(k+1)} = x_i^{(k)} + \frac{\omega}{a_{ii}}\left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij}x_i^{(k)} - a_{ii}x_i^{(k)}\right).$$

It is easy to translate the formulae above into code which requires a loop to compute the sums, a loop over i and a loop over k. Note that if $\omega = 1$, then the relaxation method reduces to Gauss-Seidel iteration. For $\omega \neq 1$, the relaxation method can increase the rate of convergence, depending on the characteristics of the system.

9.3 Example of the Iteration Method

Suppose we are required to solve

$$10x_1 - x_2 + x_3 = 20,$$

$$x_1 - 20x_2 - 2x_3 = -16,$$

$$2x_1 - x_2 + 20x_3 = 23.$$

We re-write this system as

$$x_1 = \frac{1}{10}(20 + x_2 - x_3),$$

$$x_2 = \frac{1}{20}(16 + x_1 - 2x_3),$$

$$x_3 = \frac{1}{20}(23 - 2x_1 + x_2).$$

Note that the characteristic matrix A of this system is diagonally dominant and that convergence is likely because $1/a_{ii}$ is small. Applying Jacobi iteration, we have

$$x_1^{(n+1)} = (20 + x_2^{(n)} - x_3^{(n)})/10,$$

$$x_2^{(n+1)} = (16 + x_1^{(n)} - 2x_3^{(n)})/20,$$

$$x_3^{(n+1)} = (23 - 2x_1^{(n)} + x_2^{(n)})/20,$$

and taking $\mathbf{x}^{(0)} = 0$, gives

$$\mathbf{x}^0 = (0, 0, 0)^T,$$

$$\mathbf{x}^{(1)} = (2.0, 0.8, 1.15)^T,$$

$$\mathbf{x}^{(2)} = (1.965, 1.015, 0.990)^T,$$

$$\mathbf{x}^{(3)} = (2.0025, 0.99725, 1.003)^T,$$

Applying Gauss-Seidel iteration, we get

$$x_1^{(n+1)} = (20 + x_2^{(n)} - x_3^{(n)})/10,$$

$$x_2^{(n+1)} = (16 + x_1^{(n+1)} - 2x_3^{(n)})/20,$$

$$x_3^{(n+1)} = (23 - 2x_1^{(n+1)} + x_2^{(n+1)})/20,$$

and with $\mathbf{x}^0 = 0$, we have

$$\mathbf{x}^{(0)} = (0, 0, 0)^T,$$

$$\mathbf{x}^{(1)} = (2.0, 0.9, 0.995)^T,$$

$$\mathbf{x}^{(2)} = (1.995, 0.999475, 1.000024)^T,$$

$$\mathbf{x}^{(3)} = (2.000055, 1.000050, 0.999995)^T.$$

The exact solution vector is $\mathbf{x} = (2, 1, 1)^T$.

9.4 General Formalism

Given $A\mathbf{x} = \mathbf{b}$, we decompose the characteristic matrix A into lower triangular L, diagonal D and upper triangular U matrices, i.e.

$$A = L + D + U$$

or Characteristic Matrix = Lower triangular + Diagonal + Upper triangular.

Then,

$$(L + D + U)\mathbf{x} = \mathbf{b},$$

$$D\mathbf{x} = -L\mathbf{x} - U\mathbf{x} + \mathbf{b},$$

and

$$\mathbf{x} = -D^{-1}L\mathbf{x} - D^{-1}U\mathbf{x} + D^{-1}\mathbf{b}$$

where

$$D^{-1} = \text{diag}(a_{11}^{-1}, a_{22}^{-1}, ..., a_{nn}^{-1}).$$

The iteration methods can then be written as follows:

Jacobi iteration

$$\mathbf{x}^{(n+1)} = -D^{-1}(L + U)\mathbf{x}^{(n)} + D^{-1}\mathbf{b} = M_J\mathbf{x}^{(n)} + \mathbf{c}_J$$

Gauss-Seidel iteration

$$\mathbf{x}^{(n+1)} = -D^{-1}L\mathbf{x}^{(n+1)} - D^{-1}U\mathbf{x}^{(n)} + D^{-1}\mathbf{b}$$

or after rearranging,

$$\mathbf{x}^{(n+1)} = -(D + L)^{-1}U\mathbf{x}^{(n)} + (D + L)^{-1}\mathbf{b} = M_G\mathbf{x}^{(n)} + \mathbf{c}_G$$

Relaxation iteration

$$\mathbf{x}^{(n+1)} = \mathbf{x}^{(n)} + \omega(-D^{-1}L\mathbf{x}^{(n+1)} - D^{-1}U\mathbf{x}^{(n)} + D^{-1}\mathbf{b} - \mathbf{x}^{(n)})$$

which after rearranging gives

$$\mathbf{x}^{(n+1)} = -(D + \omega L)^{-1}[(\omega - 1)D + \omega U]\mathbf{x}^{(n)} + (D + \omega L)^{-1}\omega\mathbf{b} = M_R\mathbf{x}^{(n)} + \mathbf{c}_R$$

These results are compounded in the table below.

Method	**Iteration Matrix** M	Vector $\mathbf{c}$
Jacobi	$-D^{-1}(L + U)$	$D^{-1}\mathbf{b}$
Gauss-Seidel	$-(D + L)^{-1}U$	$(D + L)^{-1}\mathbf{b}$
Relaxation	$-(D + \omega L)^{-1}[(\omega - 1)D + \omega U]$	$\omega(D + \omega L)^{-1}\mathbf{b}$

SOR .v. SUR

When do we use under ($\omega < 1$) and over ($\omega > 1$) relaxation? Experimental results suggest the following:

(i) Use $\omega > 1$ when Gauss-Seidel iteration converges monotonically (most common).

(ii) Use $\omega < 1$ when Gauss-Seidel iteration is oscillatory.

Note that as a general rule of thumb, if A is diagonally dominant, SOR or SUR iteration is only slightly faster than Gauss-Seidel iteration. However, If A is near or not diagonally dominant, then SOR or SUR can provide a noticeable improvement.

9.5 Criterion for Convergence of Iterative Methods

There are two criterion for convergence:

(i) Sufficient condition: A is strictly diagonally dominant.

(ii) Necessary and sufficient condition: The iteration matrix M has eigenvalues λ_i such that

$$\max_i \mid \lambda_i \mid < 1.$$

9.5.1 Diagonal Dominance

The matrix

$$A = \begin{pmatrix} 5 & 2 & -1 \\ 1 & 6 & -3 \\ 2 & 1 & 4 \end{pmatrix}$$

is diagonally dominant because

$$5 > 2 + \mid -1 \mid,$$

$$6 > 1 + \mid -3 \mid,$$

$$4 > 2 + 1$$

and the matrix

$$A = \begin{pmatrix} 1 & 2 & -2 \\ 1 & 1 & 1 \\ 2 & 2 & 1 \end{pmatrix}$$

is not diagonally dominant because

$$1 < 2 + \mid -2 \mid,$$

$$1 < 1 + 1,$$

$$1 < 2 + 2.$$

In general, A is diagonally dominant if

$$| a_{ii} | > \sum_{\substack{j=1 \\ j\neq i}}^{n} | a_{ij} | \; \forall i.$$

9.5.2 Sufficient Condition for Convergence

Let us define the error $\mathbf{e}^{(n)}$ at each iteration (n) as

$$\mathbf{x}^{(n)} = \mathbf{x} + \mathbf{e}^{(n)}$$

where $\mathbf{x}$ is the exact solution. Now, since

$$\mathbf{x}^{(n+1)} = M\mathbf{x}^{(n)} + \mathbf{c}$$

we have

$$\mathbf{x} + \mathbf{e}^{(n+1)} = M(\mathbf{x} + \mathbf{e}^{(n)}) + \mathbf{c}.$$

If this process is appropriate to obtain a solution, then we must have

$$\mathbf{x} = M\mathbf{x} + \mathbf{c}.$$

From the equations above, we get

$$\mathbf{e}^{(n+1)} = M\mathbf{e}^{(n)},$$

i.e.

$$\begin{aligned}
\mathbf{e}^{(1)} &= M\mathbf{e}^{(0)}, \\
\mathbf{e}^{(2)} &= M\mathbf{e}^{(1)} = M(M\mathbf{e}^{(0)}) = M^2\mathbf{e}^{(0)}, \\
\mathbf{e}^{(3)} &= M\mathbf{e}^{(2)} = M(M^2\mathbf{e}^{(0)}) = M^3\mathbf{e}^{(0)}, \\
&\vdots \\
\mathbf{e}^{(n)} &= M^n\mathbf{e}^{(0)}.
\end{aligned}$$

Now, for global convergence, we require that

$$\mathbf{e}^{(n)} \to 0 \quad \text{as} \quad n \to \infty$$

or

$$\lim_{n\to\infty} M^n\mathbf{e}^{(0)} = 0 \;\; \forall \;\; \mathbf{e}^{(0)}.$$

This will occur if

$$\|M^n\| < 1.$$

But

$$\|M^n\| \leq \|M\|^n$$

and hence, we require that

$$\|M\|^n < 1$$

or

$$\|M\| < 1.$$

Consider Jacobi iteration, where

$$M_J = -D^{-1}(L+U) = -\frac{a_{ij}}{a_{ii}}, \quad j \neq i.$$

For the ℓ_∞ norm

$$\|M_J\|_\infty = \max_i \sum_{\substack{j=1 \\ j\neq i}}^{n} \frac{| a_{ij} |}{| a_{ii} |} < 1$$

and therefore

$$| a_{ii} | > \sum_{\substack{j=1 \\ j\neq i}}^{n} | a_{ij} | .$$

Diagonal dominance is a sufficient but not a necessary condition for convergence. However, in general, the greater the diagonal dominance, the greater the rate of convergence.

9.5.3 Proof of the Necessary Condition for Convergence

For convergence, we require that

$$\lim_{n\to\infty} \mathbf{e}^{(n)} = 0.$$

Let M be the iteration matrix associated with a given process and consider the eigenvalue equation

$$(M - I\lambda_i)\mathbf{v}_i = 0$$

where λ_i are the eigenvalues of M and $\mathbf{v}_i$ are the eigenvectors of M. (N.B. Eigenvalues and eigenvectors are discussed in the following chapter.) Let us now write the error vector $\mathbf{e}^{(0)}$ as

$$\mathbf{e}^{(0)} = \sum_{i=1}^{n} a_i \mathbf{v}_i$$

where a_i are appropriate scalars. From previous results we have

$$\mathbf{e}^{(n)} = M^n \mathbf{e}^{(0)}.$$

Hence,

$$M\mathbf{e}^{(0)} = M\sum_i a_i\mathbf{v}_i = \sum_i a_i M\mathbf{v}_i = \sum_i a_i\lambda_i\mathbf{v}_i$$

since

$$M\mathbf{v}_i = \lambda_i\mathbf{v}_i.$$

Further,

$$M(M\mathbf{e}^{(0)}) = M\sum_i a_i\lambda_i\mathbf{v}_i = \sum_i a_i\lambda_i M\mathbf{v}_i = \sum_i a_i\lambda_i^2\mathbf{v}_i$$

and by induction,

$$\mathbf{e}^{(n)} = M^n \mathbf{e}^{(0)} = \sum_i a_i \lambda_i^n \mathbf{v}_i$$

Now, we want

$$\mathbf{e}^{(n)} \to 0 \quad \text{as} \quad n \to \infty.$$

But this will only occur if

$$\lambda_i^n \to 0 \quad \text{as} \quad n \to \infty,$$

a condition which requires that

$$\rho(M) < \max_i \mid \lambda_i \mid .$$

The parameter $\rho(M)$ is called the 'spectral radius'. This result leads to the following recipe for establishing the convergence of an iterative process for solving $A\mathbf{x} = \mathbf{b}$:

Given

$$\mathbf{x}^{(n+1)} = M\mathbf{x}^{(n)} + \mathbf{c},$$

(i) find the largest eigenvalue of M;

(ii) if the eigenvalue is < 1 in modulus, then the process converges;

(iii) if the eigenvalue if > 1 in modulus, then the process diverges.

Example Find the spectral radii for M_J and M_G given that

$$A = \begin{pmatrix} -4 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & -4 \end{pmatrix}.$$

The iteration matrix for Jacobi's method is

$$M_J = -D^{-1}(L+U) = \begin{pmatrix} 0 & 1/4 & 0 \\ 1/4 & 0 & 1/4 \\ 0 & 1/4 & 0 \end{pmatrix}.$$

The eigenvalues of M_J can be found by solving equation (see Chapter 10)

$$\mid M_J - \lambda I \mid = 0 \quad \text{for} \quad \lambda, \quad \text{i.e.}$$

$$\begin{vmatrix} -\lambda & 1/4 & 0 \\ 1/4 & -\lambda & 1/4 \\ 0 & 1/4 & -\lambda \end{vmatrix} = 0$$

or

$$-\lambda \begin{vmatrix} -\lambda & 1/4 \\ 1/4 & -\lambda \end{vmatrix} - \frac{1}{4} \begin{vmatrix} 1/4 & 1/4 \\ 0 & -\lambda \end{vmatrix} = 0.$$

Thus,

$$\lambda \left(\lambda^2 - \frac{1}{16} \right) - \frac{\lambda}{16} = 0$$

and

$$\lambda = 0 \quad \text{or} \quad \pm\frac{1}{\sqrt{8}}.$$

Hence,

$$\rho(M_J) = \max_i \mid \lambda_i \mid = \frac{1}{\sqrt{8}} \simeq 0.3536.$$

The Gauss-Seidel iteration matrix is given by

$$M_G = -(D+L)^{-1}U.$$

Using Jordan's method (see Chapter 7) to find $(D+L)^{-1}$ we get

$$M_G = \begin{pmatrix} 0 & 1/4 & 0 \\ 0 & 1/16 & 1/4 \\ 0 & 1/64 & 1/16 \end{pmatrix}.$$

The eigenvalues of M_G are then obtained by solving

$$\begin{vmatrix} -\lambda & 1/4 & 0 \\ 0 & 1/16-\lambda & 1/4 \\ 0 & 1/64 & 1/16-\lambda \end{vmatrix} = 0$$

giving

$$\lambda = 0, 0 \quad \text{or} \quad \frac{1}{8}.$$

Therefore

$$\rho(M_G) = 0.125.$$

9.5.4 Estimating the Number of Iterations

The smaller $\mid \lambda \mid_{\max}$, the faster $\mathbf{e}^{(n)} \to 0$ as $n \to \infty$. Hence, the magnitude of $\rho(M)$ indicates how fast the iteration processes converges. This observation can be used to estimate the number of iterations. If n is the number of iterations required to obtain a solution vector accurate to d decimal places, then

$$\rho^n \simeq 10^{-d}$$

so that n is given by

$$n \simeq -\frac{d}{\log_{10}\rho}.$$

In the SOR method, the iteration matrix is a function of the relaxation parameter ω. Hence the spectral radius ρ is a functional of ω. The optimum value of ω is the one for which ρ is a minimum. In some simple cases, the optimum value of ω can be computed analytically. However, in practice, it is common to sweep through a range of values of ω (in steps of 0.1 for example) using an appropriate computer program and study the number of iterations required to generate a given solution. The relaxation parameter ω is then set at the value which gives the least number of iterations required to produce a solution with a given accuracy.

9.5.5 The Stein-Rosenberg Theorem

In general, $\rho(M_G) \leq \rho(M_J)$. Therefore Gauss-Seidel iteration usually converges faster than Jacobi iteration for those cases in which both methods of iteration can be applied. The comparisons between Jacobi and Gauss-Seidel iteration are compounded in the Stein-Rosenberg theorem:

If M_J contains no negative elements, then there exists the following four possibilities:

Convergence

(i) $\rho(M_J) = \rho(M_G) = 0$;

(ii) $0 < \rho(M_G) < \rho(M_J) < 1$.

Divergence

(iii) $\rho(M_J) = \rho(M_G) = 1$,

(iv) $1 < \rho(M_J) < \rho(M_G)$.

9.6 The Conjugate Gradient Method

The conjugate gradient method can be used to solve $A\mathbf{x} = \mathbf{b}$ when A is symmetric positive definite. We start by defining the term 'conjugate'. Two vectors $\mathbf{v}$ and $\mathbf{w}$ are conjugate if $\mathbf{v}^T A\mathbf{w} = 0$, $\mathbf{v}^T A\mathbf{v} > 0 \forall \mathbf{v}$. The basic idea is to find a set of linearly independent vectors $(\mathbf{v}_1, \mathbf{v}_2, ..., \mathbf{v}_n)$ which are conjugate with respect to A, i.e.

$$\mathbf{v}_i^T A\mathbf{v}_j = 0 \quad \text{if} \quad i \neq j.$$

If we expand $\mathbf{x}$ as a linear series of basis vectors

$$\mathbf{x} = \sum_{j=1}^{n} \alpha_j \mathbf{v}_j$$

then the problem is reduced to finding $\alpha_j \forall j$. Now, the equation $A\mathbf{x} = \mathbf{b}$ becomes

$$\sum_{j=1}^{n} \alpha_j A\mathbf{v}_j = \mathbf{b}.$$

Multiplying both sides of this equation by $\mathbf{v}_i^T$ we get

$$\sum_{j=1}^{n} \alpha_j \mathbf{v}_i^T A\mathbf{v}_j = \mathbf{v}_i^T \mathbf{b}.$$

Now, since $\mathbf{v}_i^T A\mathbf{v}_j = 0$ if $i \neq j$, we have

$$\sum_{j=1}^{n} \alpha_j \mathbf{v}_i^T A\mathbf{v}_j = \alpha_i \mathbf{v}_i^T A\mathbf{v}_i = \mathbf{v}_i^T \mathbf{b}.$$

Thus,

$$\alpha_i = \frac{\mathbf{v}_i^T\mathbf{b}}{\mathbf{v}_i^T A\mathbf{v}_i}$$

and the solution becomes

$$\mathbf{x} = \sum_{j=1}^{n}\left(\frac{\mathbf{v}_i^T\mathbf{b}}{\mathbf{v}_i^T A\mathbf{v}_i}\right)\mathbf{v}_i.$$

This solution only requires the computation of $A\mathbf{v}_i$ and the scalar products. Therefore, the computation is easy once the conjugate directions $\mathbf{v}_i$ are known. In principal, the conjugate gradient method is a direct method. However, in practice, an iterative solution is required to find the conjugate directions using a 'Gram-Schmidt' type process.

9.6.1 The Gram-Schmidt Process

Our problem is, given $(\mathbf{v}_1, \mathbf{v}_2, ..., \mathbf{v}_n)$ of a vector space R^n with inner product denoted by $\langle\ \ \rangle$, find an orthonormal basis $(\mathbf{w}_1, \mathbf{w}_2, ..., \mathbf{w}_n)$ of R^n. The Gram-Schmidt process is based on the following procedure:

$$\mathbf{w}_1 = \mathbf{v}_1,$$

$$\mathbf{w}_2 = \mathbf{v}_2 - \frac{\langle\mathbf{v}_2, \mathbf{w}_1\rangle}{\|\mathbf{w}_1\|^2}\mathbf{w}_1,$$

$$\mathbf{w}_3 = \mathbf{v}_3 - \frac{\langle\mathbf{v}_3, \mathbf{w}_2\rangle}{\|\mathbf{w}_2\|^2}\mathbf{w}_2 - \frac{\langle\mathbf{v}_3, \mathbf{w}_1\rangle}{\|\mathbf{w}_1\|^2}\mathbf{w}_1,$$

$$\vdots$$

$$\mathbf{w}_n = \mathbf{v}_n - \frac{\langle\mathbf{v}_n, \mathbf{w}_{n-1}\rangle}{\|\mathbf{w}_{n-1}\|^2}\mathbf{w}_{n-1} - ... - \frac{\langle\mathbf{v}_n, \mathbf{w}_2\rangle}{\|\mathbf{w}_2\|^2}\mathbf{w}_2 - \frac{\langle\mathbf{v}_n, \mathbf{w}_1\rangle}{\|\mathbf{w}_1\|^2}\mathbf{w}_1,$$

where

$$\langle\mathbf{x}, \mathbf{y}\rangle \equiv \mathbf{x}^T\mathbf{y}, \quad \|\mathbf{x}\| \equiv \sqrt{\langle\mathbf{x}, \mathbf{x}\rangle}.$$

The orthonormal basis is then given by

$$\left(\frac{\mathbf{w}_1}{\|\mathbf{w}_1\|}, \frac{\mathbf{w}_2}{\|\mathbf{w}_2\|}, ..., \frac{\mathbf{w}_n}{\|\mathbf{w}_n\|}\right),$$

i.e.

$$\langle\mathbf{w}_i/\|\mathbf{w}_i\|, \mathbf{w}_j/\|\mathbf{w}_j\|\rangle = 0, \quad i \neq j.$$

9.6.2 Example of the Gram-Schmidt Process

Consider

$$\mathbf{v}_1 = (1, 1, 0)^T \text{ and } \mathbf{v}_2 = (-1, 0, 1)^T.$$

Then

$$\mathbf{w}_1 = \mathbf{v}_1 = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix},$$

$$\mathbf{w}_2 = \mathbf{v}_2 - \frac{\langle \mathbf{v}_2, \mathbf{v}_1 \rangle}{\|\mathbf{w}_1\|^2}\mathbf{w}_1$$

$$= \begin{pmatrix} -1 \\ 0 \\ 1 \end{pmatrix} - \frac{(-1)}{2}\begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} -1 \\ 0 \\ 1 \end{pmatrix} + \begin{pmatrix} 1/2 \\ 1/2 \\ 0 \end{pmatrix} = \begin{pmatrix} -1/2 \\ 1/2 \\ 1 \end{pmatrix},$$

$$\mathbf{w}_1^T\mathbf{w}_2 = 0,$$

$$\mathbf{w}_2^T\mathbf{w}_1 = 0.$$

Therefore $\mathbf{w}_1$ and $\mathbf{w}_2$ are orthogonal and the orthonormal basis is

$$\frac{\mathbf{w}_1}{\|\mathbf{w}_1\|} = \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \\ 0 \end{pmatrix}, \quad \frac{\mathbf{w}_2}{\|\mathbf{w}_2\|} = \frac{2}{3}\begin{pmatrix} -1/2 \\ 1/2 \\ 1 \end{pmatrix}.$$

9.6.3 Practical Algorithm for the Conjugate Gradient Method

To solve $A\mathbf{x} = \mathbf{b}$, we begin with an arbitrary starting vector $\mathbf{x}_0 = (1, 1, ..., 1)^T$ say and compute the residual $\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0$. We then set $\mathbf{v}_0 = \mathbf{r}_0$ and feed the result into the following iterative loop:

(i) compute coefficients of conjugate gradients

$$\alpha_j = \frac{\mathbf{v}_i^T\mathbf{r}_i}{\mathbf{v}_i^T A\mathbf{v}_i};$$

(ii) update $\mathbf{x}$,

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \alpha_i\mathbf{v}_i;$$

(iii) compute the new residual

$$\mathbf{r}_{i+1} = \mathbf{r}_i + \alpha_i A\mathbf{v}_i;$$

(iv) compute the next conjugate gradient using the Gram-Schmidt process

$$\mathbf{v}_{i+1} = \mathbf{r}_i + \frac{\mathbf{r}_{i+1}^T\mathbf{r}_{i+1}}{\mathbf{r}_i^T\mathbf{r}_i}\mathbf{v}_i.$$

Important points

1. The conjugate directions are $\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2, ..., \mathbf{v}_n$.
2. The matrix A is only used to compute $A\mathbf{v}_i$.
3. The process is stopped when $\|\mathbf{r}_i\| \leq \epsilon$, i.e. the required tolerance.
4. The algorithm works well when A is large and sparse.

We conclude this chapter with a schematic diagram illustrating the principal iterative methods of approach to solving $A\mathbf{x} = \mathbf{b}$ as given below.

$A\mathbf{x} = \mathbf{b}$ where A is sparse and symmetric or non-symmetric	$A\mathbf{x} = \mathbf{b}$ where A is sparse, symmetric and positive definite
$\downarrow$	$\downarrow$
$\rho(\lambda) < 1$	Conjugate gradient method
$\downarrow$	$\downarrow$
SOR with $\omega = 1$	$\mathbf{x}$
$\swarrow \searrow$	
Monotonic Oscillatory convergence convergence	
$\downarrow$ $\downarrow$	
SOR SUR	
$\downarrow$ $\downarrow$	
$\mathbf{x}$ $\mathbf{x}$	

Diagram 9.1: Schematic diagram illustrating the iterative methods

9.7 Summary of Important Results

Iterative Method

$$\mathbf{x}^{(n+1)} = M\mathbf{x}^{(n)} + \mathbf{c}$$

Jacobi Iteration

$$M = -D^{-1}(L+U), \quad \mathbf{c} = D^{-1}\mathbf{b}$$

Gauss-Seidel Iteration

$$M = -(D+L)^{-1}U, \quad \mathbf{c} = (D+L)^{-1}\mathbf{b}$$

Relaxation Iteration

$$M = -(D+\omega L)^{-1}[(\omega-1)D+\omega U], \quad \mathbf{c} = \omega(D+\omega L)^{-1}\mathbf{b}$$

SOR Method

$$1 < \omega < 2$$

SUR Method

$$0 < \omega < 1$$

Condition for Convergence

$$||M|| << 1$$

Sufficient Condition for Convergence

$$| a_{ii} | > \sum_{\substack{j=1 \\ j \neq i}}^{n} | a_{ij} |$$

Necessary Condition for Convergence

$$\rho(M) < \max_i | \lambda_i |$$

Conjugate Gradient Solution

$$\mathbf{x} = \sum_{j=1}^{n} \left(\frac{\mathbf{v}_i^T \mathbf{b}}{\mathbf{v}_i^T A \mathbf{v}_i} \right) \mathbf{v}_i, \qquad \mathbf{v}_i^T A \mathbf{v}_j = 0 \text{ if } i \neq j.$$

9.8 Further Reading

- Traub J F, *Iterative Methods for the Solution of Equations*, Prentice-Hall, 1964.
- Smith G D, *Numerical Solution of Partial Differential Equations*, Oxford University Press, 1978.
- Fogeil M, *The Linear Algebra Problem Solver*, Research and Education Association, 1986.
- Gerald C F and Weatley P O, *Applied Numerical Analysis*, Addison-Wesley, 1989.
- Ciarlet O G, *Introduction to Numerical Linear Algebra and Optimization*, Cambridge University Press, 1989.

9.9 Problems

9.1 (i) Solve the following system of equations using Gauss-Seidel iteration:

$$-2x_1 + x_2 = -1,$$

$$x_1 - 2x_2 + x_3 = 0,$$

$$x_2 - 2x_3 + x_4 = 0,$$

$$x_3 - 2x_4 = 0.$$

Observe the number of iterations that are required for the process to converge, giving a solution that is correct to 2 decimal places.

(ii) Compare the number of iterations using Jacobi and then Gauss-Seidel method that are required to solve $A\mathbf{x} = (2,4,6,8)^T$ where

$$A = \begin{pmatrix} 4 & 1 & 0 & 0 \\ 1 & 4 & 1 & 0 \\ 0 & 1 & 4 & 1 \\ 0 & 0 & 1 & 2 \end{pmatrix}.$$

9.2 Consider the following equations:

$$5x_1 + 2x_2 - x_3 = 6,$$

$$x_1 + 6x_2 - 3x_3 = 4,$$

$$2x_1 + x_2 + 4x_3 = 7.$$

State whether or not the characteristic matrix of this system is diagonally dominant. Compute the Jacobi and Gauss-Seidel iteration matrices for this system and solve these equations using Jacobi iteration and then Gauss-Seidel iteration. Use a 'stopping criterion' given by

$$\max_i \mid x_i^{(n+1)} - x_i^{(n)} \mid \leq \frac{1}{2} \times 10^{-2}.$$

If we wanted to improve the rate of convergence of the Gauss-Seidel process, should we use SOR or SUR. By choosing a suitable value for the relaxation parameter, solve the system of equations above using the relaxation method. Is the rate of convergence significantly improved? Comment on your result.

9.3 Compare the convergence rates of GS and SOR iteration using a relaxation parameter value of 1.6 to solve the equations:

$$-3x_1 + x_2 + 3x_4 = 1,$$

$$x_1 + 6x_2 + x_3 = 1,$$

$$x_2 + 6x_3 + x_4 = 1,$$

$$3x_1 + x_3 - 3x_4 = 1.$$

Use a 'stopping criterion' of 5×10^{-3}. Comment on the result.

9.4 For which of the following system of equations does the (i) Jacobi method and (ii) Gauss-Seidel method converge? Start with the zero vector and use a tolerance of 0.001:

(a)

$$-4x_1 + x_2 = 1,$$

$$x_1 - 4x_2 + x_3 = 1,$$

$$x_2 - 4x_3 + x_4 = 1,$$

$$x_3 - 4x_4 = 1.$$

(b)

$$2x_1 + x_2 + x_3 = 4,$$
$$x_2 + 2x_2 + x_3 = 4,$$
$$x_1 + x_2 + 2x_3 = 4.$$

(c)

$$x_1 + 2x_2 - 2x_3 = 3,$$
$$x_1 + x_2 + x_3 = 0,$$
$$2x_1 + 2x_2 + x_3 = 1.$$

9.5 Find the spectral radii of the Jacobi and Gauss-Seidel iteration matrices corresponding to each system given in question 9.4. Hence, comment on the convergence or divergence in each case.

9.6 Which, if any, of the coefficient matrices in question 9.4 are diagonally dominant? Is diagonal dominance a necessary condition for convergence of the Jacobi and Gauss-Seidel method?

9.7 $\|\mathbf{v}\|$ denotes any norm of R^n and $\|A\|$ is the associated norm for an $n \times n$ matrix A. Prove that if $\rho(A)$ denotes the spectral radius of A, then

$$\rho(A) \leq \|A\|.$$

9.8 Find a set of vectors which are mutually conjugate with respect to the matrix

$$A = \begin{pmatrix} 3 & 1 & -1 \\ 1 & 2 & 1 \\ -1 & 1 & 3 \end{pmatrix}.$$

Hence, solve the equation

$$A\mathbf{x} = (4, 6, 4)^T.$$

9.9 Use the Gram-Schmidt process to solve the equation $A\mathbf{x} = (4 \ \ 4 \ \ 4)^T$ where

$$A = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 3 & -1 \\ 0 & -1 & 2 \end{pmatrix}.$$